Fundamentals of Software Engineering

TAHIR FAROOQ

FAST-NU

Agenda – What will you Learn Today?

What is Software Architecture?

Architecture Attributes

Software Architecture

What is Software Architecture?

The Software Architecture of a system consists of a description of the system elements, interactions between the system elements, patterns that guide the system elements and constraints on the relationships between system elements.

[Shaw and Garlan]

Carnegie Mellon University

What is Software Architecture?

The software architecture of a program or computing system is the structure or structures of the system, which comprise software components, the externally visible properties of those components, and the relationships among them.

[SEI]

What is Software Architecture?

Architectural design: The process of defining a collection of hardware and software components and their interfaces to establish the framework for the development of a computer system.

[IEEE Glossary]

Importance of Architecture

- **Communication between stakeholders**
  - A high-level presentation of the system
  - Use for understanding, negotiation and communication
- **Early design decisions**
  - Profound effect on the systems quality attributes, e.g. **performance**, **availability**, **maintainability** etc.
- **Large-scale reuse**
  - If similar system have common requirements, modules can be identified and reused

Architecture Attributes

Performance

Localize the operations to minimize subsystem communication

Scale up …

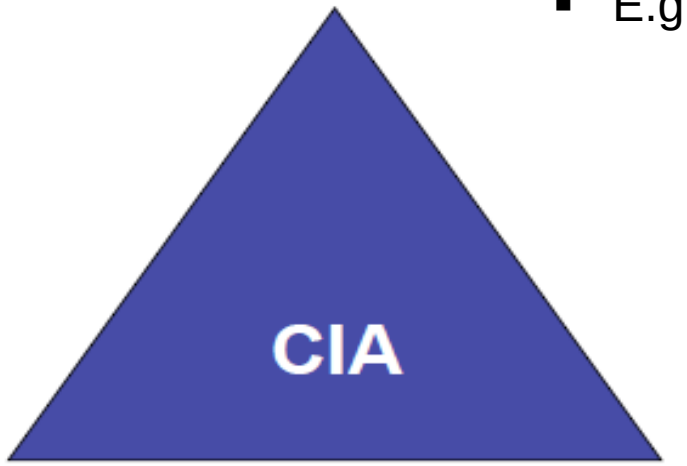
Scale out …

Security

Use a layered architecture with critical assets in inner layers

Confidentiality

- Only authorized users can read the information
- E.g. Military



Availability

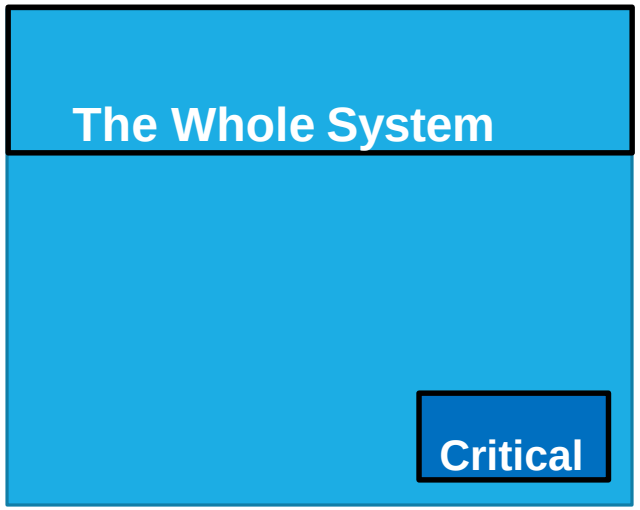
- Right information is available at the right time
- Important for everyone

Integrity

- Only authorized users can modify, edit or delete data.
- E.g. bank systems

Safety

Isolate safe-critical components



Design so that all safety critical operations are located in one or few modules / subsystems.

Availability

Include redundant components in the architecture

Maintainability

Use fine-grain, self-contained components

System structuring and Design Process

System structuring

- Concerned with decomposing the system into interacting sub-systems

- The architectural design is normally expressed as a **block diagram** presenting an overview of the system structure

- More specific models showing how sub-systems share data, are distributed and interface with each other may also be developed

Modular decomposition

The identified sub-system are decomposed into modules.

Control Modelling

A model of the control relationships between the different parts of the system id established.

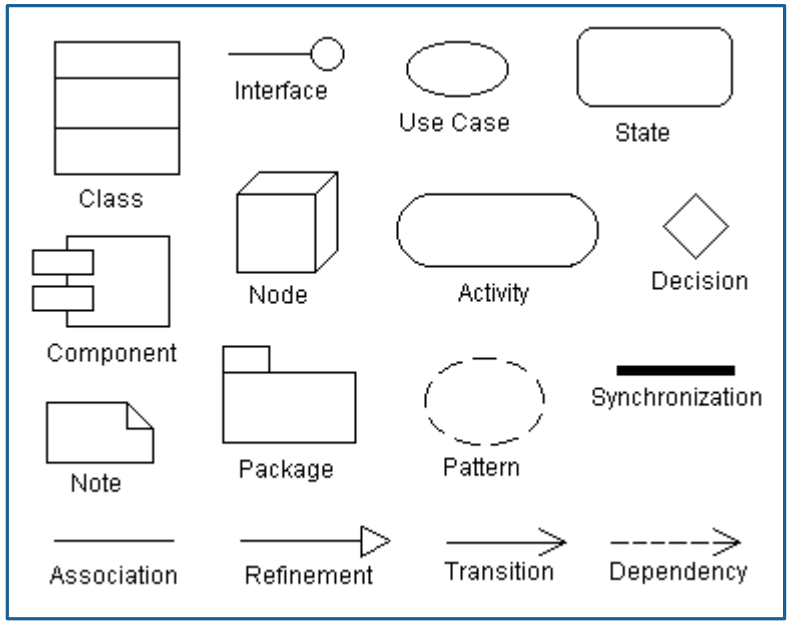
Unified Modeling Language

UML for Software Architecture

The Unified Modeling Language (UML) is a **graphical language** for

- visualizing (pictorially connected components)
- specifying,
- constructing, and
- documenting

the ***artifacts*** of a software-intensive system.



UML offers a **standard way** to draw a **system's blueprints**.

Same notation for every project

Architecture View Models

- A model is a complete, simplified description of a system from a particular perspective or viewpoint.

- There is no single view that can present all aspects of complex software to stakeholders!!!

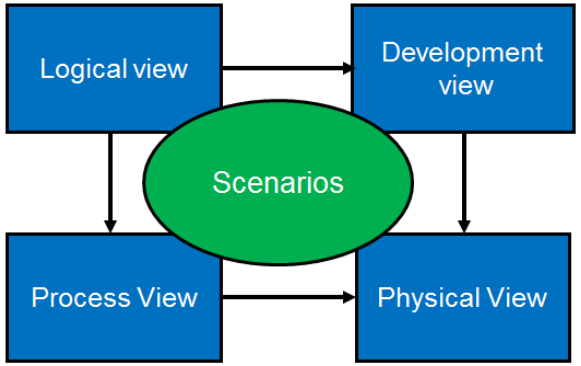
View Model

- View models **provide** partial representations of the software architecture to specific stakeholders such as
  - the system users,
  - the analyst/designer,
  - the developer/programmer,
  - the system integrator, and
  - the system engineer.

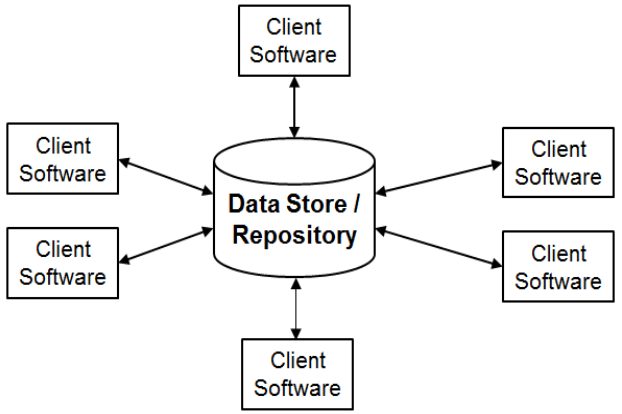
View Model

Software designers can organize the description of their architecture decisions in different views.

Krutchen's 4+1 View Model of Software Architecture



Architecture Styles



Krutchen’s 4+1 View Model

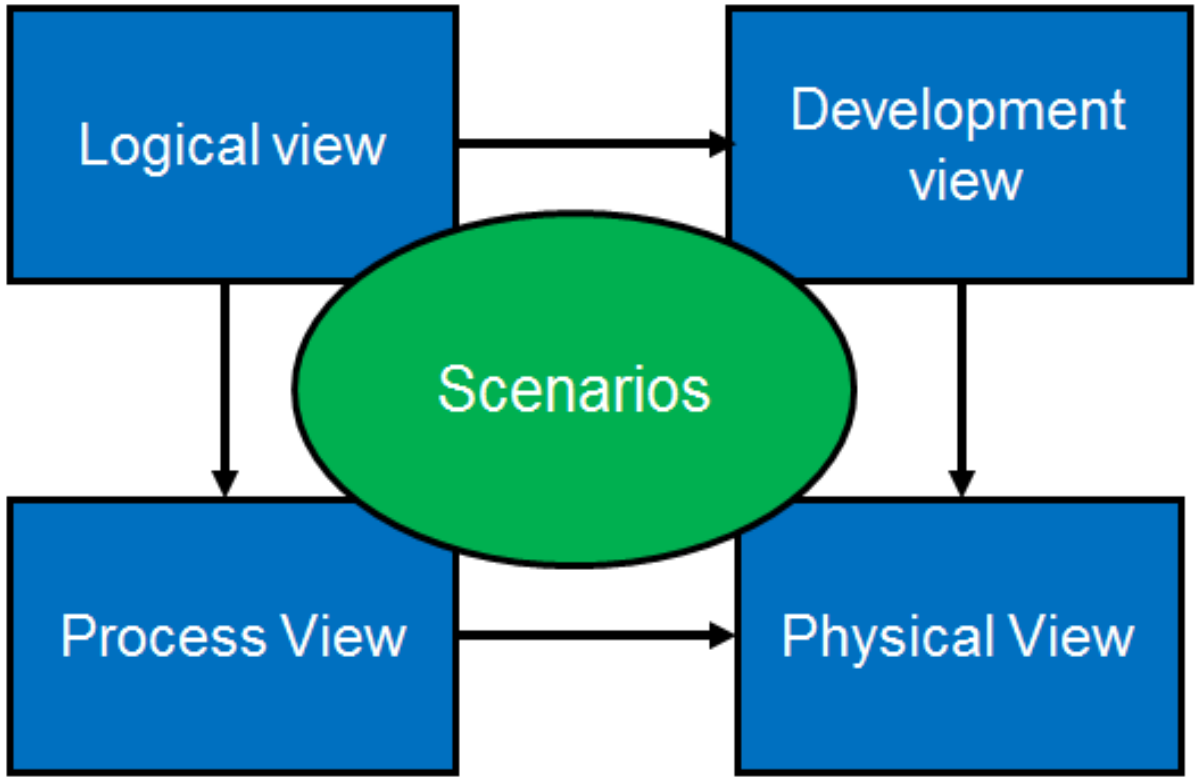
The 4 +1 View Model

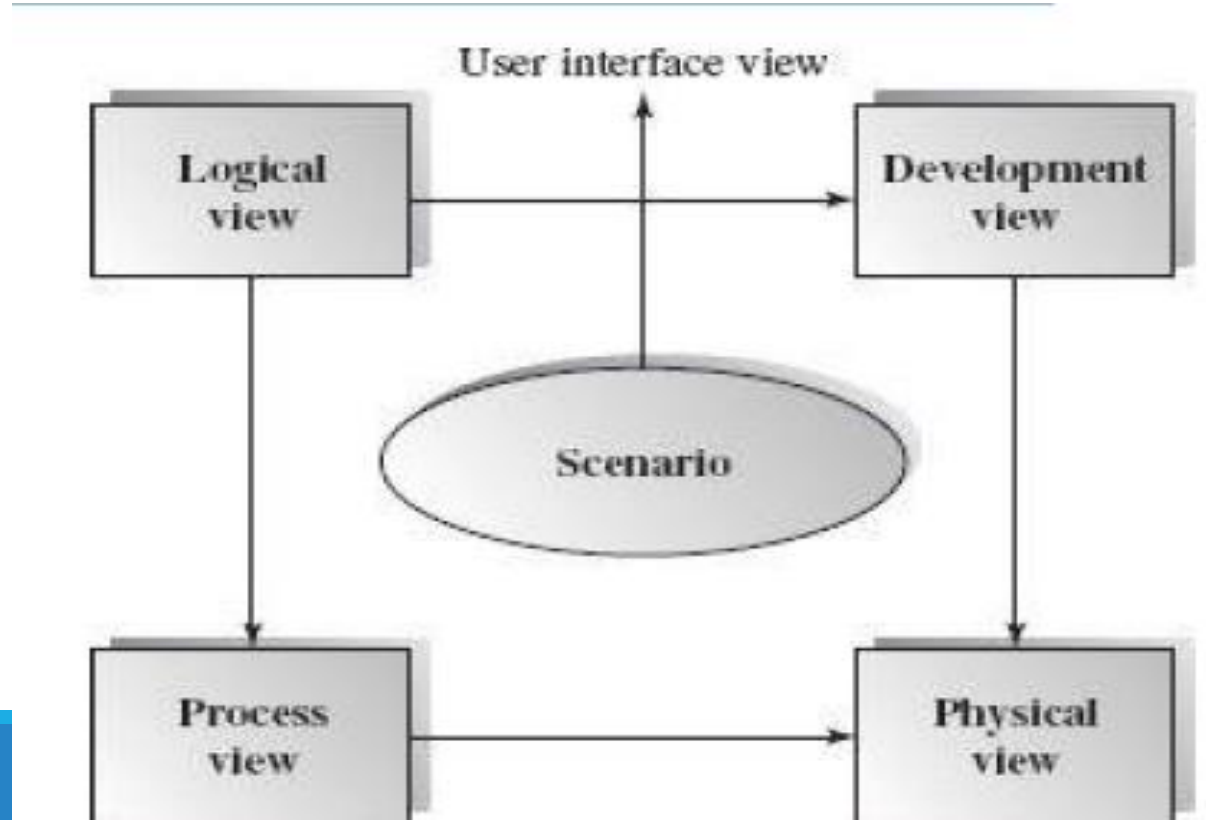
- The 4+1 view model was originally introduced by Philippe Kruchten (Kruchten, 1995).

- The model provides four essential views:
  - the logical view,
  - the process view,
  - the physical view, and
  - the development view
- and fifth is the scenario view

The 4+1 view model

- **Multiple-view model** that addresses **different aspects and concerns** of the system. Standardizes the software design documents and makes the design easy to understand by all stakeholders.



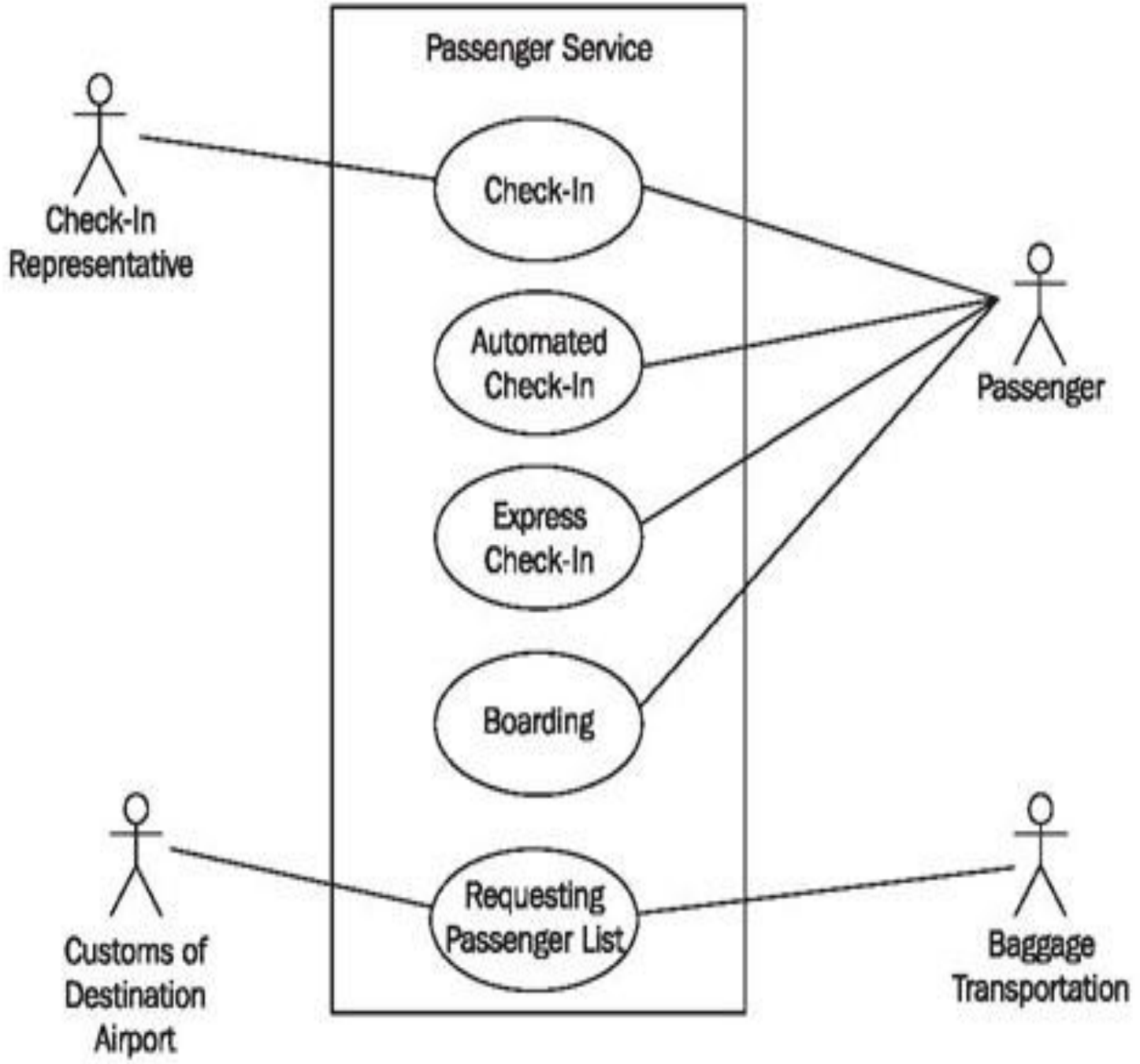


The Scenario View

- The scenario **view describes the functionality** of the system, i.e., how **the user employs the system** and **how the system provides services to the users.**

- It helps designers to discover architecture elements during the design process and to validate the architecture design afterward.

  - The UML use case diagram and other verbal documents


Passenger Service
Check-In Representative
Check-In
Automated Check-In
Express Check-In
Boarding
Passenger
Customs of Destination Airport
Requesting Passenger List
Baggage Transportation

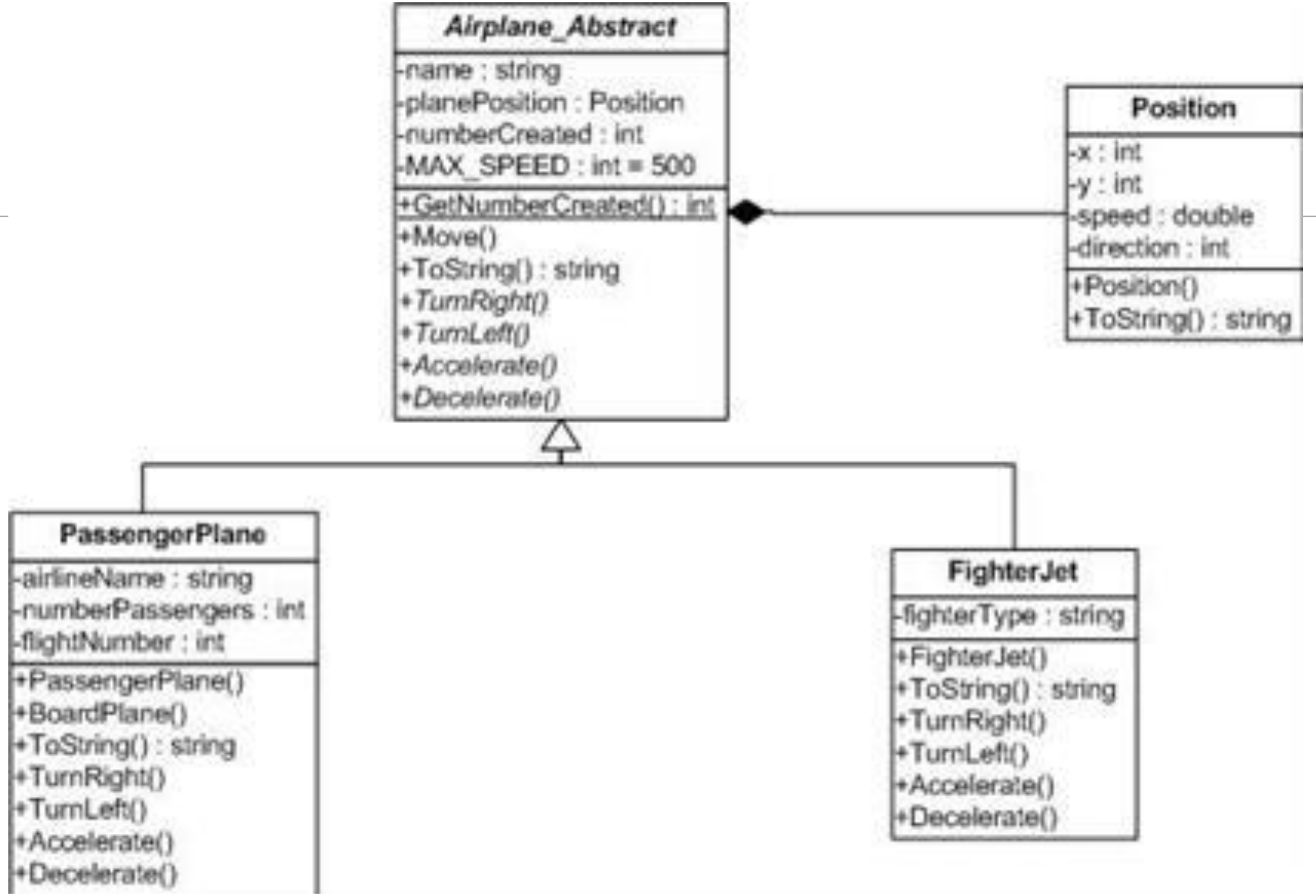
The Logical or Conceptual View

The logical view is **based on application domain entities** necessary to implement the functional requirements.

The logical view specifies system decomposition into conceptual entities (such as objects) and connections between them (such as associations).

UML is concept of OO so therefore

- The logical view is typically supported by
  - UML **static diagrams** including class diagrams and
  - UML **dynamic diagrams** such as the interaction overall diagram, **sequence diagram**, **communication diagram**, **state diagram**, and **activity diagram**.


Airplane_Abstract
-name : string
-planePosition : Position
-numberCreated : int
-MAX_SPEED : int = 500
+GetNumberCreated() : int
+Move()
+ToString() : string
+TurnRight()
+TurnLeft()
+Accelerate()
+Decelerate()
Position
-x : int
-y : int
-speed : double
-direction : int
+Position()
+ToString() : string
PassengerPlane
-airlineName : string
-numberPassengers : int
-flightNumber : int
+PassengerPlane()
+BoardPlane()
+ToString() : string
+TurnRight()
+TurnLeft()
+Accelerate()
+Decelerate()
FighterJet
-fighterType : string
+FighterJet()
+ToString() : string
+TurnRight()
+TurnLeft()
+Accelerate()
+Decelerate()

The Development or Module View

- The development view derives from the logical view and describes the static organization of the system modules.

- Modules such as namespaces, class library, subsystem, or packages are building blocks that group classes for further development and implementation.

- UML diagrams such as **package diagrams** and **component diagrams** are often used to support this view.

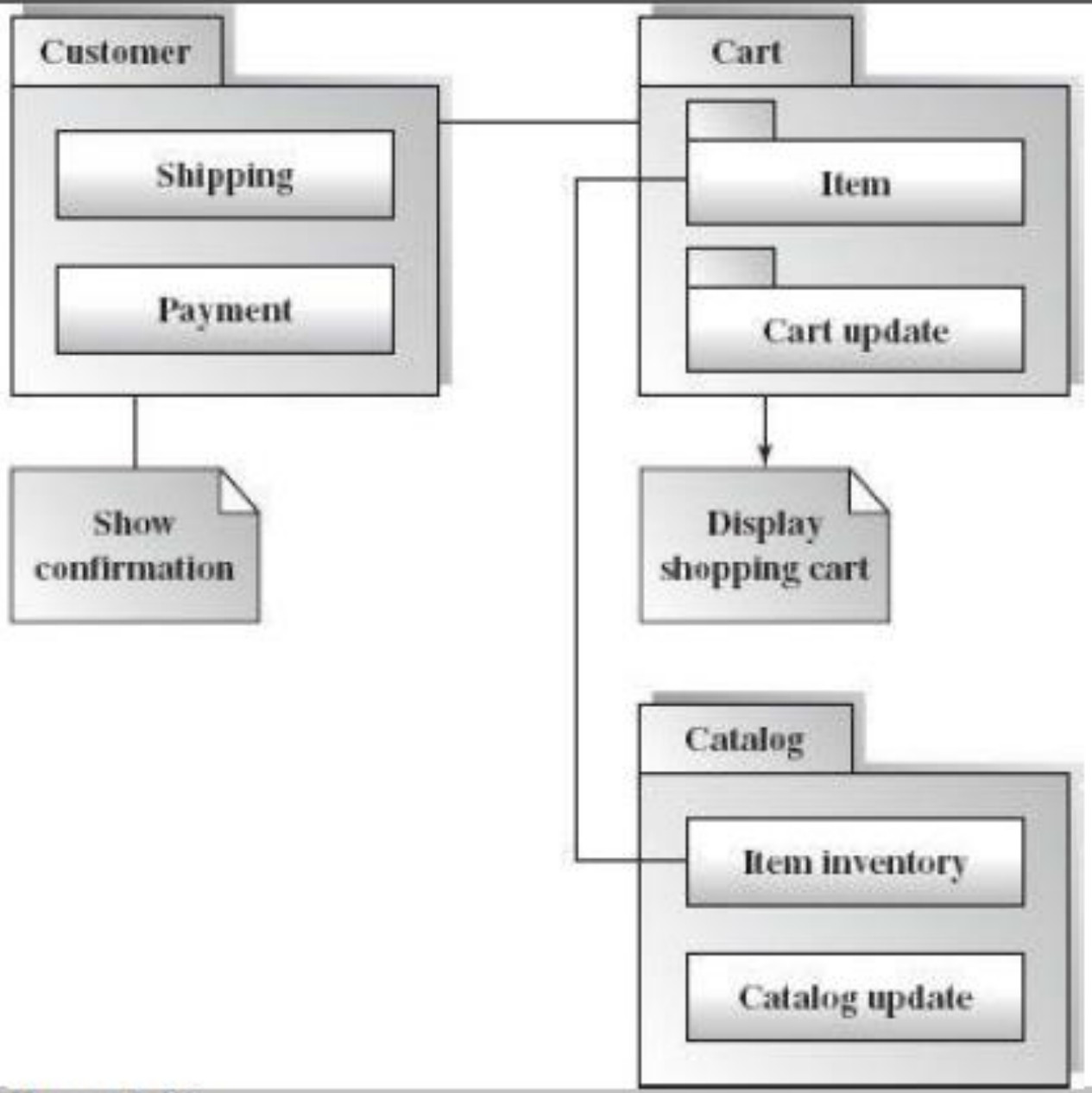


Figure 3.17
Package diagram in the development view

Show static behavior

The Process View

- The process view focuses on the dynamic aspects of the system, i.e., its **execution time behavior.**
- This view maps functions, activities, and interactions onto runtime implementation.
- The process view takes care of the concurrency and synchronization issues between subsystems.
- The UML **activity diagram** and interaction overview diagram support this view.

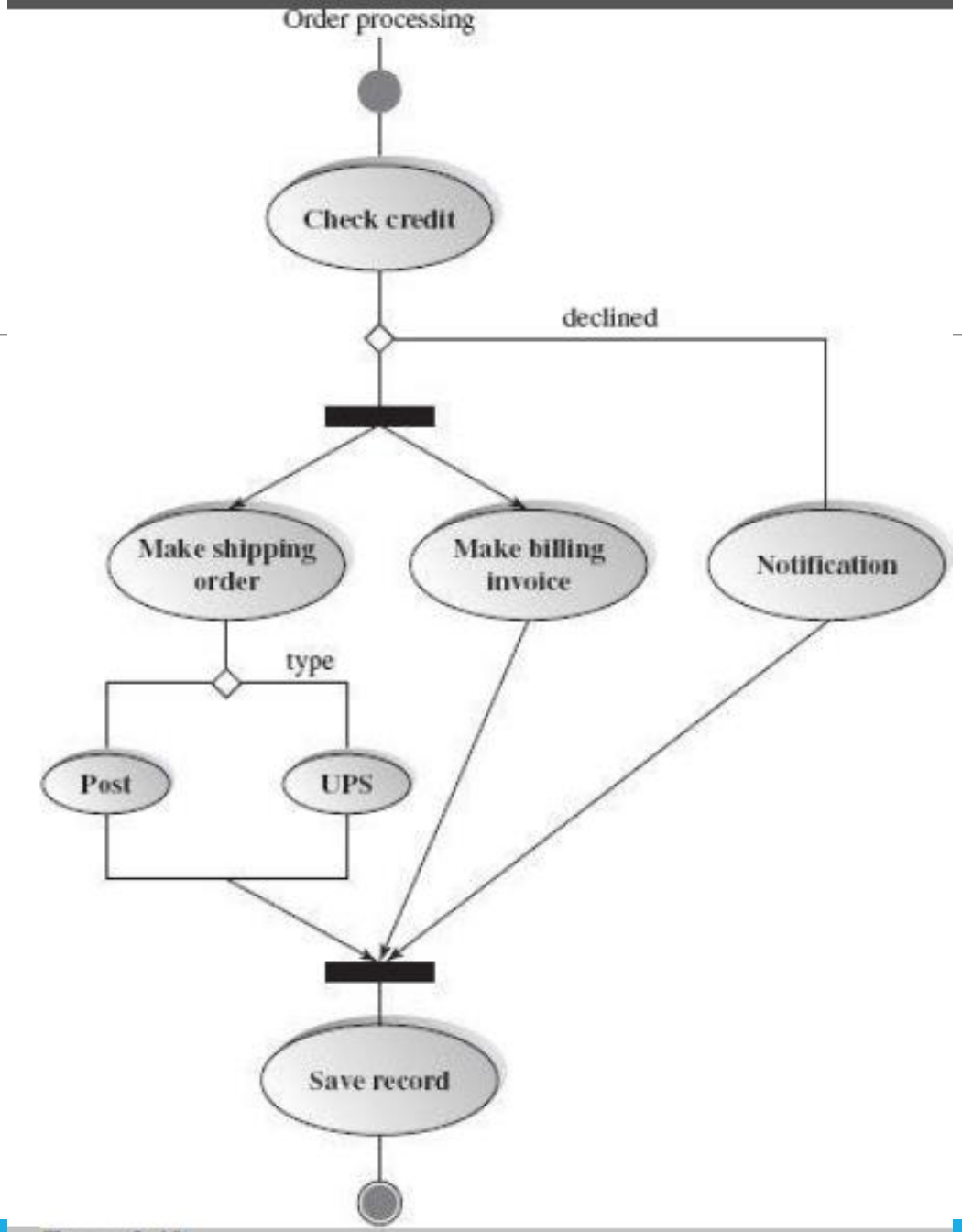


Figure 3.18
Activity diagram in the process view

The Physical View

- The physical view **describes installation**, **configuration**, and **deployment** of the software application.
- It concerns itself with how to deliver the deploy-able system.
- The physical view shows the mapping of software onto hardware.
  - It is particularly of interest in distributed or parallel systems.
- The UML **deployment diagrams** and other documentation are often used to support this view.

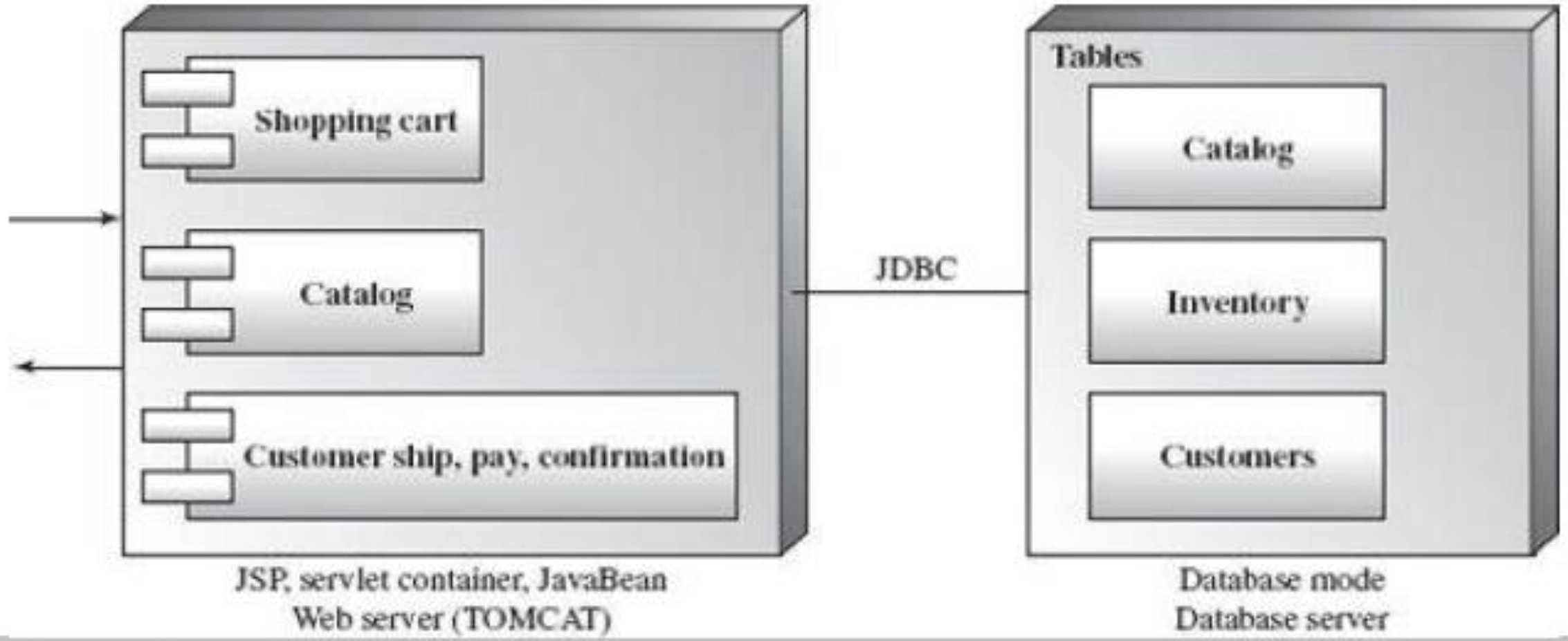


Figure 3.19
Deployment diagram in the physical view

The User Interface View

- The User Interface (UI) view is an extended view that provides a clear user-computer interface view and hides implementation details.
- This view may be provided as a series of screen snapshots or a dynamic, interactive prototype demo.
- Any modification on this view will have direct impact on the scenarios view.

Architectural Styles

Architectural Styles

- The architectural model of a system may conform to a generic architectural model or style

- An awareness of these styles can simplify the problem of defining system architectures

Architectural Styles (cont.)

- Each style describes a system category that encompasses:
  1) A set of components (e.g. database, computational modules) that perform a function required by a system
  2) A set of connectors that enable "communication, coordination and cooperation" among components
  3) Constraints that define how components can be integrated to form the system

Recap

What is Software Architecture?

Architecture Attributes